

Two Pointers in Python — Questions + Answers

A study guide with the actual LeetCode problem statements, approach, Python solutions, examples, and complexity.

How to use this guide

For each problem, first read only the **Question**. Try to solve it yourself on paper or in VS Code. Then check the **Approach** and **Answer**. The goal is to understand why each pointer moves, not to memorize code.

Two Pointers — Pattern 1

The first pattern to learn is two pointers moving inward from opposite ends:

```
left = 0
right = len(arr) - 1

while left < right:
    # use arr[left] and arr[right]

    left += 1
    right -= 1
```

Key question: After looking at the current left and right values, which pointer can I safely move, and why?

LC 1 — Two Sum

QUESTION

Given an array of integers `nums` and an integer `target`, return the indices of the two numbers such that they add up to `target`. You may assume that each input has exactly one solution, and you may not use the same element twice.

Example: `nums = [2, 7, 11, 15]`, `target = 9` → `[0, 1]`

Try it yourself before reading below.

APPROACH

This problem is commonly solved with a hash map rather than Two Pointers because the input is not guaranteed to be sorted. As you scan the array, store each value's index. For the current value `x`, look for `target - x` in the map.

PYTHON ANSWER

```
def twoSum(nums, target):
    seen = {}

    for i, x in enumerate(nums):
        need = target - x

        if need in seen:
            return [seen[need], i]

        seen[x] = i

    return []
```

COMPLEXITY: $O(n)$ time and $O(n)$ extra space.

WHAT TO LEARN: Important: 'Two Sum' does not automatically mean Two Pointers. Pattern selection depends on the input and its properties.

LC 125 — Valid Palindrome

QUESTION

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Return True if the given string is a palindrome, otherwise return False.

Example: "A man, a plan, a canal: Panama" → True; "race a car" → False

Try it yourself before reading below.

APPROACH

Put left at the beginning and right at the end. Skip non-alphanumeric characters. Compare the characters case-insensitively. If they differ, return False. Otherwise move both pointers inward.

PYTHON ANSWER

```
def isPalindrome(s):
    left = 0
    right = len(s) - 1

    while left < right:
        while left < right and not s[left].isalnum():
            left += 1

        while left < right and not s[right].isalnum():
            right -= 1

        if s[left].lower() != s[right].lower():
            return False

        left += 1
        right -= 1

    return True
```

COMPLEXITY: O(n) time and O(1) extra space.

WHAT TO LEARN: This is the cleanest first example of opposite-end Two Pointers.

LC 167 — Two Sum II: Input Array Is Sorted

QUESTION

Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers that add up to a specific target. Return their 1-based indices. You may assume exactly one solution exists.

Example: numbers = [2, 7, 11, 15], target = 9 → [1, 2]

Try it yourself before reading below.

APPROACH

Start left at the smallest value and right at the largest. If the sum is too small, increase left because the array is sorted. If the sum is too large, decrease right. If it equals target, return the indices.

PYTHON ANSWER

```
def twoSum(numbers, target):
    left = 0
    right = len(numbers) - 1

    while left < right:
        total = numbers[left] + numbers[right]

        if total == target:
            return [left + 1, right + 1]
        elif total < target:
            left += 1
        else:
            right -= 1

    return []
```

COMPLEXITY: $O(n)$ time and $O(1)$ extra space.

WHAT TO LEARN: This is the classic rule: sorted array + pair target → consider opposite-end Two Pointers.

LC 344 — Reverse String

QUESTION

Write a function that reverses a string represented as a list of characters. You must modify the input list in-place with $O(1)$ extra memory.

Example: ['h','e','l','l','o'] $\rightarrow$ ['o','l','l','e','h']

Try it yourself before reading below.

APPROACH

Place one pointer at each end. Swap the two characters, then move both pointers inward. Stop when the pointers meet or cross.

PYTHON ANSWER

```
def reverseString(s):
    left = 0
    right = len(s) - 1

    while left < right:
        s[left], s[right] = s[right], s[left]

        left += 1
        right -= 1
```

COMPLEXITY: $O(n)$ time and $O(1)$ extra space.

WHAT TO LEARN: This is the simplest mechanical example of the opposite-end pattern.

LC 11 — Container With Most Water

QUESTION

You are given an integer array `height` where `height[i]` represents the height of a vertical line at position `i`. Choose two lines that, together with the x-axis, form a container that holds the most water. Return the maximum amount of water.

Example: `height = [1,8,6,2,5,4,8,3,7]` → 49

Try it yourself before reading below.

APPROACH

Start with the widest container: `left` at the beginning and `right` at the end. Area is `width × the shorter height`. The shorter wall limits the water, so move the pointer at the shorter wall. Moving the taller wall cannot improve the limiting height while width gets smaller.

PYTHON ANSWER

```
def maxArea(height):
    left = 0
    right = len(height) - 1
    best = 0

    while left < right:
        width = right - left
        current = min(height[left], height[right]) * width
        best = max(best, current)

        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return best
```

COMPLEXITY: $O(n)$ time and $O(1)$ extra space.

WHAT TO LEARN: The key is not just moving pointers—it is proving why one pointer can be discarded.

LC 15 — 3Sum

QUESTION

Given an integer array `nums`, return all the unique triplets `[nums[i], nums[j], nums[k]]` such that `i`, `j`, and `k` are different indices and `nums[i] + nums[j] + nums[k] == 0`. The answer must not contain duplicate triplets.

Example: `[-1,0,1,2,-1,-4] → [[-1,-1,2],[-1,0,1]]`

Try it yourself before reading below.

APPROACH

Sort the array. Fix one number `nums[i]`. Then use `left = i + 1` and `right = n - 1` to find two numbers whose sum is `-nums[i]`. Because the array is sorted, move `left` when the total is too small and `right` when it is too large. Skip duplicates.

PYTHON ANSWER

```
def threeSum(nums):
    nums.sort()
    result = []

    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        left = i + 1
        right = len(nums) - 1

        while left < right:
            total = nums[i] + nums[left] + nums[right]

            if total == 0:
                result.append([nums[i], nums[left], nums[right]])

                left += 1
                right -= 1

                while left < right and nums[left] == nums[left - 1]:
                    left += 1

                while left < right and nums[right] == nums[right + 1]:
                    right -= 1

            elif total < 0:
                left += 1
            else:
                right -= 1

    return result
```

COMPLEXITY: $O(n^2)$ time after sorting; auxiliary space is $O(1)$ apart from the returned result, with sorting-space details depending on the implementation.

WHAT TO LEARN: 3Sum is an important progression: an outer loop chooses one element, and Two Pointers solves the remaining 2Sum problem.

LC 42 — Trapping Rain Water

QUESTION

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much rainwater can be trapped after it rains.

Example: $[0,1,0,2,1,0,1,3,2,1,2,1] \rightarrow 6$

Try it yourself before reading below.

APPROACH

For each position, trapped water depends on the tallest wall to its left and right. Maintain `left_max` and `right_max`. When `height[left] <= height[right]`, the left side is the limiting side, so process the left position using `left_max`. Otherwise process the right position.

PYTHON ANSWER

```
def trap(height):
    left = 0
    right = len(height) - 1
    left_max = 0
    right_max = 0
    water = 0

    while left < right:
        if height[left] <= height[right]:
            if height[left] >= left_max:
                left_max = height[left]
            else:
                water += left_max - height[left]
            left += 1
        else:
            if height[right] >= right_max:
                right_max = height[right]
            else:
                water += right_max - height[right]
            right -= 1

    return water
```

COMPLEXITY: $O(n)$ time and $O(1)$ extra space.

WHAT TO LEARN: This is the more advanced Two Pointer problem in the set because you maintain information about both sides.

Practice Order

Order	Problem	Main skill
1	LC 344 — Reverse String	Learn pointer movement and swapping.
2	LC 125 — Valid Palindrome	Learn skipping and comparing from both ends.
3	LC 167 — Two Sum II	Learn the sorted-array movement rule.
4	LC 11 — Container With Most Water	Learn how to justify which pointer moves.
5	LC 15 — 3Sum	Combine an outer loop with Two Pointers.
6	LC 42 — Trapping Rain Water	Learn a more advanced invariant.
7	LC 1 — Two Sum	Compare Two Pointers with the Hash Map approach.

Final checklist

1. Is the array sorted?
2. Can I use one pointer at each end?
3. What does left represent?
4. What does right represent?
5. What condition tells me to move left?
6. What condition tells me to move right?
7. Why is that pointer safe to move?
8. When do I stop?
9. What are the time and space complexities?

The most important habit: whenever you move a pointer, be able to explain why the discarded position can never be part of a better answer.